12 · Exam day — procedure and one-page summary

Read this the night before and once more outside the lab. It doesn't replace typing the programs from memory (chapter 13).

12.1 The 6-step routine

1. **Read the question twice.** Is it the plain experiment or an ASSIGNMENT twist (descending order, insert at beginning, circular queue…)? Find the matching "If the examiner changes the question" section in your head.
2. **Write the record first** (aim, algorithm, complexity). It takes 5 minutes, earns marks even if the code fights you, and plans your program.
3. **Type the skeleton, compile it, then grow it.** `#include <iostream>` / `using namespace std;` / `int main() { return 0; }`. Build once (F9 / F11) so you know the setup works. Then add one function at a time and compile after each.
4. **Fix errors top-down.** Fix only the first error and recompile (chapter 1.6).
5. **Run the manual's sample input** and check the output against the expected answer below.
6. **Test one edge case** (empty list, bad position, overflow). Examiners like seeing it handled.

12.2 Expected outputs (the manual's sample inputs)

Exp	Input	Output
1	—	Value = 25, address = pointer value, value via pointer = 25
2	—	10 → 20 → 10 → 30; addresses base, +4, base, +8
3	10 20 30 40 50; begin 5 / pos 3 val 25 / end 60	5 10 20 30 40 50 / 10 20 25 30 40 50 / 10 20 30 40 50 60
4	10 20 30 40 50; begin / pos 3 / end	20 30 40 50 / 10 20 40 50 / 10 20 30 40
5	[[1,2],[3,4]] × [[5,6],[7,8]]	19 22 / 43 50
6	[[1,2,3],[4,5,6]]	1 4 / 2 5 / 3 6
7	5 2 4 6 1 3	1 2 3 4 5 6
8	10 7 8 9 1 5	1 5 7 8 9 10
9	push 10 20 30, pop	pops 30 (LIFO)
10	enqueue 10 20 30, dequeue	display 10 20 30; deletes 10; then 20 30
11	insert 10 20 30, delete	10 20 30; deletes 10; then 20 30
12	insert 10 20 30, delete	10 20 30; deletes 10; then 20 30

12.3 Complexity cheat sheet

Operation	Time	Space
Pointer access / arithmetic (Exp 1–2)	O(1)	O(1)
Array insert / delete at beginning or position (Exp 3–4)	O(n)	O(1)
Array insert / delete at end	O(1)	O(1)
Matrix multiply (r1·c1·c2) / n × n	$O(n^3)$	$O(n^2)$
Transpose m × n	O(m·n)	O(m·n)
Insertion sort: best / avg / worst	O(n) / $O(n^2)$ / $O(n^2)$	O(1)
Quick sort: best / avg / worst	O(n log n) / O(n log n) / $O(n^2)$	O(log n) avg, O(n) worst
Bubble / selection sort (worst)	$O(n^2)$	O(1)
Merge sort (all cases)	O(n log n)	O(n)
Stack push / pop / peek	O(1)	O(MAX)
Queue enqueue / dequeue; display	O(1); O(n)	O(MAX)
Singly list: insert end / delete begin / traverse	O(n) / O(1) / O(n)	O(n)
Circular list: insert end / delete begin / traverse	O(n) / O(n) / O(n)	O(n)

12.4 The lines that decide each program

Exp	The line(s) that matter
1	`ptr = &number;` · `*ptr`
2	`int *ptr = arr;` · `ptr++` moves one int (4 bytes)
3	`for (i = n; i >= pos; i--) arr[i] = arr[i-1];` · `arr[pos-1] = value;` · `n++`
4	`for (i = pos-1; i < n-1; i++) arr[i] = arr[i+1];` · `n--`
5	`C[i][j] = 0;` then `C[i][j] += A[i][k] * B[k][j];` for k < c1 · check `c1 == r2`
6	`T[j][i] = A[i][j];` · display `(T, cols, rows)`
7	`while (j >= 0 && arr[j] > key) { arr[j+1] = arr[j]; j--; }` · `arr[j+1] = key;`
8	`if (arr[j] < pivot) { i++; swap(arr[i], arr[j]); }` · `swap(arr[i+1], arr[high]); return i+1;`
9	full: `top == MAX-1` · empty: `top == -1`
10	full: `rear == MAX-1` · empty: `front == -1` · `if (front == rear) front = rear = -1;`
11	`while (temp->next != NULL)` · `head = head->next; delete temp;` · `temp = temp->next;`
12	`while (temp->next != head)` · `do { … } while (temp != head);` · `last->next = head;`

12.5 General viva questions

Question	Answer
What is a data structure?	A way of organising data in memory so operations on it are efficient (array, stack, queue, linked list, tree, graph).
Linear vs. non-linear?	Linear: elements in a sequence (array, stack, queue, list). Non-linear: hierarchy or network (tree, graph).
Static vs. dynamic?	Static: size fixed at compile time (array). Dynamic: grows at run time with `new` (linked list).
What does O(n) mean?	The running time grows in proportion to n (the input size), ignoring constants.
Time vs. space complexity?	Time = how the number of steps grows with n; space = how much extra memory grows with n.
Algorithm vs. program?	An algorithm is the step-by-step method (language-independent); a program is that method written in a language.
What is recursion?	A function calling itself on a smaller problem, with a base case that stops it (quick sort, merge sort, reverse display).
Why `using namespace std;`?	To use `cout`, `cin`, `endl` without writing `std::` each time.